

Twenty Years On: Rebuilding a Robocode Robot with Agents

A steel thread from college craft to the RoboRumble top 15, and what the human still does

Leigh Griffin*

Draft, 8 October 2026

Abstract

Robocode was, for a generation of computing students, the first program that fought back: a tank written in Java that had to sense, move, aim and survive against someone else’s code. It taught event-driven design, geometry, testing and, above all, measuring before believing. Twenty years later we return to it with agentic software engineering. Over twelve days, a multi-agent project of Claude Code sessions, steered by one developer, rebuilt the robot Hadur through 14 releases and over 100 merged pull requests, traced to about 210 EARS requirements and gated by a headless bench. Hadur rose from 64th to 13th of about 1,216 robots in the 1v1 RoboRumble, and entered the melee and team ladders (19th and 12th). We chart that history as a *steel thread*: one repeated loop of goal, staged plan, requirements, implementation, bench gate, release and live reading. We report what accelerated, where the agents’ own measurements were wrong, and which decisions stayed with the human. Our claim is modest and, we think, useful: the skills Robocode taught have not changed. What changed is where the developer spends them.

1 Introduction

Robocode [14] has been used to teach programming and artificial intelligence since IBM released it in 2001 [8, 15]. Its appeal is that the feedback is immediate and unarguable: a robot either survives the round or it does not. Underneath the game, a student meets the core of the craft. There is an event loop with a strict time budget per turn, physics to model exactly, noisy sensors, an adversary that adapts, and a public ladder, the RoboRumble [18], that ranks every entry against every other over thousands of battles. Many of us wrote our first serious Java there.

This paper is about returning to that robot twenty years later with a different way of working. Hadur began as a robot that evolved by feel (Section 2). In September and October 2026 it was rebuilt almost entirely by Claude Code agents working in a multi-agent project: planning sessions wrote staged plans, implementation sessions wrote the code, tests and bench reports, and one human set the goals, made the calls and acted in the world on the project’s behalf. The result climbed from 64th to 13th of about 1,216 robots in the 1v1 RoboRumble in ten days (Fig. 6).

The rank is not the point, though it keeps us honest. The point is the thesis behind the project: *the skills that Robocode taught have not changed, and agentic development can still proceed, accelerate and learn when those skills are applied*

deliberately. Requirements still have to be stated so they can be tested. Architecture still decides what can be tested at all. A claim still needs a measurement, and a measurement still needs a control. What an agent changes is the cost of the typing between those decisions, which falls close to zero, and so the decisions themselves become the work.

We make three contributions.

1. A **steel thread** for agentic development of a measured system: one loop, repeated for every stage, from goal to live reading (Section 3). It was used unchanged across six plans and about forty stages.
2. A **history with the data**: the architecture’s evolution, the bench results and the live ladder, each traceable to a file in the public repository [7] (Sections 4 and 5).
3. **Insights on the human’s role**, drawn from where the agents’ plans and measurements were right, where they were wrong, and who noticed (Sections 7 and 8).

2 Background

Robocode and the rumble. A Robocode battle runs in discrete turns. Each turn a robot receives events (a scan, a hit, a wall) and issues orders (turn, move, aim, fire, radar). Bullets cost energy and refund three times their power on a hit; a robot that runs out of energy is disabled. A robot cannot see bullets, only the opponent’s energy, so it must infer when the opponent fired from a drop in energy. The top robots *wave surf*: they treat each inferred shot as an expanding circle and move to where the opponent’s gun is least likely to aim [19]. The RoboRumble pairs every robot with every other and scores each pairing by average percentage score (APS), the share of the pair’s total score; a robot’s rating is the mean over about 1,215 pairings. There are separate ladders for 10-robot melee and 5-robot teams.

Hadur 1.x. Hadur was a dual-mode duel and melee robot. Its later releases (1.17 to 1.20, ported to a Maven build in May 2026) used wave surfing, k-nearest-neighbour guess-factor guns with 13 features, an anti-surfer gun and bullet shadows. It was strong but opaque. It was also not hand-written in the old sense: 26 of its May commits carry a Claude co-author line. It was built with one assistant in single sessions, by feel, and its own commit messages report progress as rounds won against Shadow (“about 49%”). It had three weaknesses that shaped everything after: nothing measured it, so a gain could not be told from noise; strategy was fused with the engine, so tactics could not be unit tested or replayed; and its picture of the enemy was noisy, because it surfed waves that never existed.

*Corresponding author. Source, requirements, bench reports and data: https://github.com/lgriffin/Hadur_Robot.

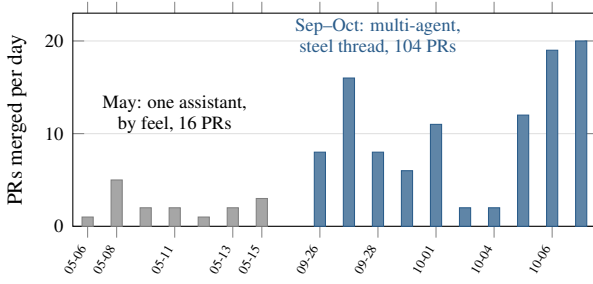


Figure 1: Pull requests merged per day. May (grey): Hadur 1.17 to 2.0 with one assistant and no bench. September to October (blue): the multi-agent project and the steel thread. Days without merges are omitted.

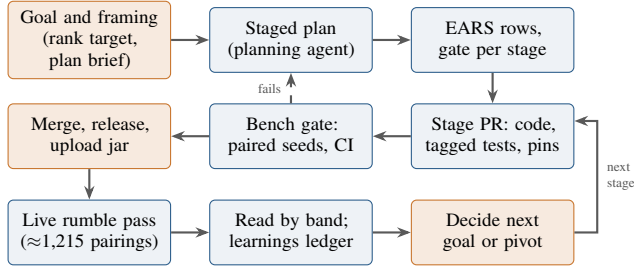


Figure 2: The steel thread. Every stage ran the loop from plan to gate; each release, which often bundled several stages, ran the rest. Orange steps are the human’s; blue steps were done by agents. A failed gate returns to the plan, not to the code.

The agentic setting. The rebuild ran in a Claude Code project: a shared space where a coordinator agent watches the project chat and starts a *thread session* for each piece of work. Each thread is a separate agent with its own repository checkout, and all share a project memory and a file store. Over the twelve days, 33 threads ran. As a standing rule, a larger model wrote the staged plans and a faster model implemented the stages. Every change reached the repository as a pull request, and an automated reviewer commented on each one. The human, the author, posted requests in the project chat, answered decision cards, and did the things only a person could do (Section 8).

Two eras, one assistant. Fig. 1 shows both eras. The assistant was Claude in both. What changed in September was the way of working: a bench before any change, requirements with IDs, an architecture that could be replayed, and a multi-agent project that could run several stages at once. The rate of merged work rose about fivefold, from 1.6 to 8.7 pull requests a day, and, more importantly, every merge after S0 came with a measurement.

3 The steel thread

A steel thread is the thinnest end-to-end path through a system that exercises every layer, in the spirit of the walking skeleton [6]. Ours is a path through the *process*, not the code: one loop that every stage follows, from a goal to a reading of the live ladder (Fig. 2). It was set at the start and then not changed, which is what made the pace safe.

Goal and plan. Each plan began with a goal stated by the human: a rank (top 30, then top 15, then top 10), a capability (a robot that remembers each opponent), or a design brief (one identity across three ladders). A planning agent turned it into stages, each small enough for one pull request and each with a *gate*: a bench result that must hold before the stage merges. Plans were written as shareable documents and copied into the repository when they started, so that the plan and its record could later be compared.

Requirements. Every behaviour is an EARS requirement [12]: “*When an enemy energy drop is not explained by the ledger, the robot shall create a wave of the remaining power.*” EARS suits agents well. Its fixed shapes (ubiquitous, event-driven, state-driven, unwanted behaviour) leave little room to be vague, and each row carries an ID (WAVE-1, RAM-3) that the tests must name. The repository holds about 210 distinct IDs.

Implementation. An implementation agent builds one stage per pull request. The rule is that each requirement due by the current stage has a Cucumber scenario tagged with its ID. Pure logic gets unit tests and jqwik property tests [4]. A change in recorded play must be explained, and the replay fixtures re-recorded by name.

Gate. The bench runs the real Robocode engine headless, one battle per process, with fixed seeds. A gate compares a candidate jar with a baseline jar on the same seeds and opponents (paired), and reports a mean difference with a 95% interval. A stage merges when its gate holds; when it does not, the stage goes back to the plan, or merges with the miss written down (Section 7).

Release and live reading. Releases are cut by a workflow, and the human enters the jar on the rumble’s participants page. A full live pass takes an afternoon. The saved pages are parsed into the repository’s data archive, read by opponent rank band, and every finding goes into a learnings ledger with its evidence and a status (confirmed, open or refuted). Findings that need a decision about the bench itself get a 5 Whys record in a kaizen register [20].

Keeping it honest. Table 1 lists the layers that stop the documents, tests and code from drifting apart. None is new. Together they let an agent change a codebase of about 37,000 lines of Java many times a day while a human reads only the summaries.

4 Architecture

Hexagonal core. The first stage after the baseline (S1) moved every line of strategy into a core with no Robocode imports, without changing how the robot played. A thin adapter turns engine events into plain records (BotInput); the core returns BotOrders; and telemetry and the profile store leave through ports [5]. A guard wraps the core: if it throws, the robot still receives safe orders and the fault is counted. The port was proved faithful by replay. Recorded battles against the reference opponents replay through a fresh

Table 1: The layers that keep strategy, requirements and code aligned. The first seven fail the build; the last two are process gates, read before a merge or after a release, and a release can go ahead of its bench gate (3.8.5 did).

Layer	What it guards
EARS rows	What the robot must do, stage by stage
Traceability test	Every requirement due by the current stage names a test; unknown tags fail
ArchUnit	No Robocode, randomness, clock, threads or I/O in the core
Property tests	Physics equals the engine's; the ledger recovers exact shot power
Cucumber	One scenario per requirement, tagged with its ID
Replay	13 recorded battles reproduce the live robot's orders bit for bit
Source pins	Hashes of each strand's packages; a change must re-pin by name
Bench gate	Paired score difference against the previous release
Live ladder	The rumble's full pass, read by band

core to the live robot's exact orders. Comparing same-seed battles turn by turn against 1.20 exposed two slips in the port, an angle read in degrees rather than radians and a gun-heat read one shot late, and both were fixed before anything else changed.

This split is what made agentic work safe. An agent can change the gun and know within minutes whether recorded play moved, and whether it moved only where the stage said it would.

One identity, three strands. By robot 3.5.1 the core had grown a problem that was about ownership, not speed. HadurCore was 2,100 lines and about 90 fields, two thirds of them duel state. Melee was a delegate, a team was impossible, because in a team battle a teammate reads as an opponent, and every new role would add branches to every event handler. The human's brief for the next plan was short:

"I want an architecture evolution for Hadur using my hexagonal approach. There are three goals: 1v1, melee and team. All are attainable. [...] It knows its role, it adapts instantly, and it does not have to guess or switch."

The planning agent turned that into six stages (A0 to A5) and five tests taken from the brief (Fig. 3). The battle's facts fix a *charter* at tick 0. A pure function, the role resolver, picks the role from the charter and three counts, and within a round it may only step down a ladder: Team, then Melee, then Duel. Every role the charter allows is built and warmed up before the first tick. The melee tracker became the kernel's single *World*, which teammates can also feed. Every package, output channel, memory file and requirement has exactly one owner, enforced by per-strand source hashes. The duel's nine packages did not change in code from A1 to A4, and no replay fixture was re-recorded. The evolution shipped as 3.7 with a second jar, a five-robot team.

Memory within a sandbox. Robocode gives each robot 200 KB of disk, forbids renaming files, and charges for every byte written but refunds nothing on delete. Hadur keeps one

profile per opponent lineage, written with a CRC and a copy-then-delete protocol, and tests kill a save at every byte of the file. The bench found that the delete protocol leaked quota (fixed by emptying a file before deleting it, which the engine does refund). Live play later found something the bench could not, which we return to in Section 7.

5 History

Table 2 summarises the six plans and Fig. 4 places them in time. Each followed the loop of Fig. 2.

Measure first (S0 to S6). The first stage changed nothing in the robot. It built the bench and measured 1.20: against Shadow 3.83c, the reference duelist, Hadur won about half its rounds but took only 40.8% of the score. Score share, not rounds won, became the headline metric. The stages that followed made the robot more correct before they made it stronger (Fig. 5). S2 replaced "any energy drop is a shot" with an energy ledger that accounts for every other cause of a drop. Scored against the engine's real bullets, it found 99.4% of the shots a scan could reveal with no false waves, and explained away 1,820 drops that 1.20 had surfed as phantom shots. S3 and S4 gave the robot a crash-safe memory and let it adapt to a remembered opponent, with tiers named only when an Agresti-Coull margin [1] was tight enough. None of these moved the score outside the noise, and the documents said so at each stage. S6 did. Bullet shadows, the firing angles that our own bullets block, raised the share to 57.4%. It took three rounds of fixing the geometry against the engine's truth logs (35%, then 83%, then 100% of destroyed enemy bullets inside a computed shadow) before the feature was trusted.

The live ladder disagrees (3.0 to 3.4). 3.0 entered the rumble at 85.5 APS over its first 268 pairings and fell to 77.2 over the next 239; 3.1 slid the same way. The bench, at every CPU setting tried, could not reproduce it. The learnings ledger recorded the gap as open and named the difference: the bench forked a fresh JVM per battle and met at most 60 opponents, whereas a rumble client runs hundreds of battles in one process with a data directory that grows by one profile per opponent. A session bench built to match (R7) traced the slide to the round-end profile save, which scanned that growing directory and caused skipped turns. 3.4 fixed it and held 85.9 APS, 20th.

A route to the top, and a leak (3.7 to 3.9). With the architecture in place, the next plan aimed at DrussGT, a long-time leader. The bench showed both robots' guns hitting below break-even against each other, so the robot firing lighter bullets keeps its lead; D1 taught the duel that, and the gate rose 12 points against DrussGT. 3.8 shipped five such stages. Live, it gained 5.2 points per pairing against the top 10, where it was aimed, and fell to 29th. Against opponents ranked 21st and below it lost 1.55 points per pairing on average, mostly through lost rounds, and with about 1,200 such pairings that outweighed the gain at the top. 3.8.5 scoped the route to duels the robot was not already winning on the guns and recovered a fifth of the loss. A bisect of releases on the opponents that had beaten 3.4 then put the remaining loss in 3.5's rammer escape, not in the new route. 3.9 tries the escape for two

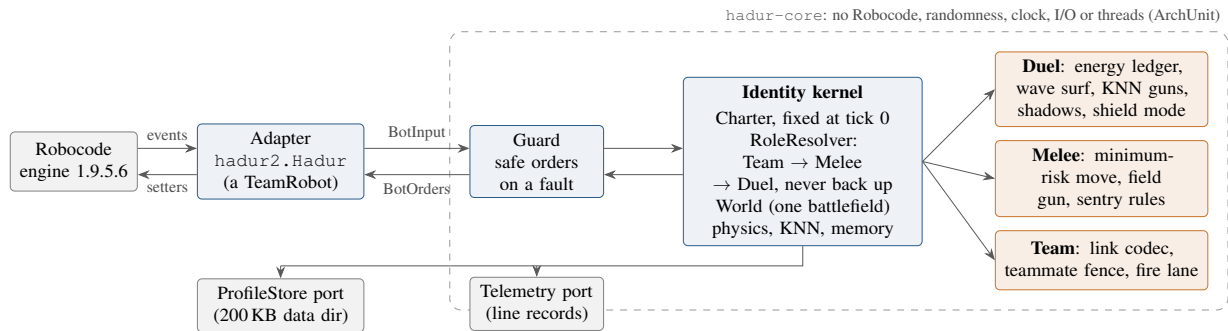


Figure 3: Hadur 3.9. The outer hexagon (adapter, guard, ports) dates from S1; the identity kernel and its three strands from A1 to A5. The dashed boundary is enforced by ArchUnit on every build.

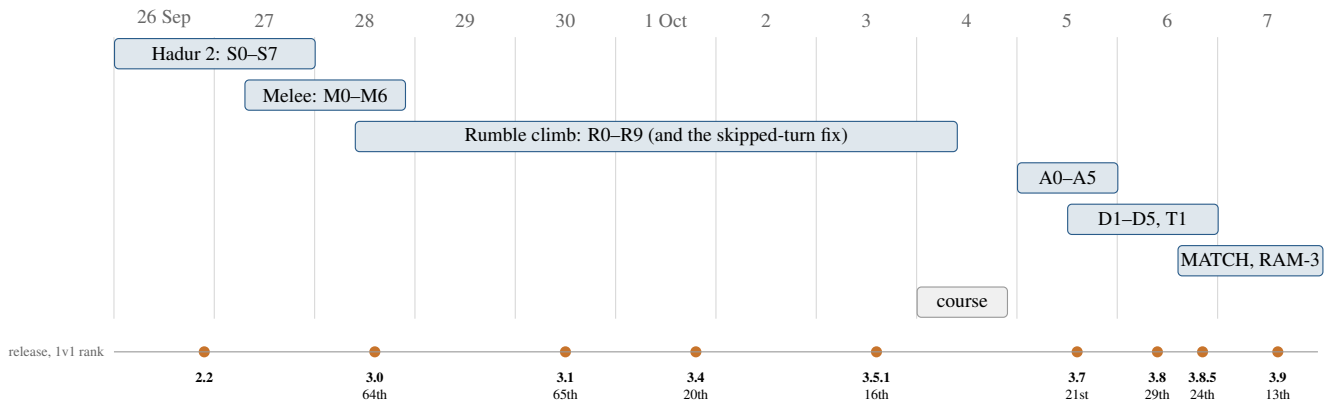


Figure 4: The twelve days. Bars are plans (grey: the companion course); dots are releases with the 1v1 rank of their first full pass. 2.2 was entered before the ladder was being read.

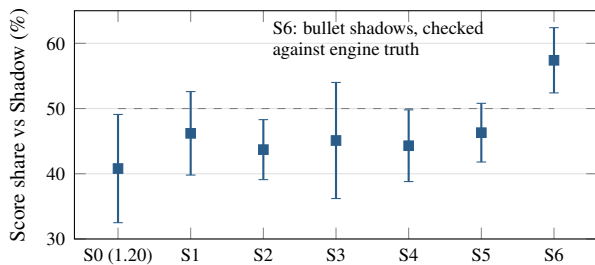


Figure 5: Score share against Shadow 3.83c by stage (cold bench, 35 rounds $\times$ 5 seeds, 95% interval). Five stages of correctness work held the score level; S6 moved it.

rounds and then keeps, per opponent, whichever of escape or fight paid. It reached 87.15 APS, Hadur's best, 13th, and 0.56 below 10th (Fig. 7).

Melee and team. The melee extension (M0 to M6) raised the reference-field APS from 37.9 to between 52.7 and 54.6, and firsts on a challenge field from 33 to 72 of 100. The team baseline (A5) fielded five robots that shared sightings; its bench found about 440 teammate collisions per round. T1 made teammates visible to each other's movement and added a fence that brakes before a collision. On the gate, collisions fell from 101,322 to 65 and score share rose from 28.8% to 48.2%. Live, the team went from 32nd of 46 to 12th, and the melee entry has held between 16th and 19th of about 415 since 3.5.1.

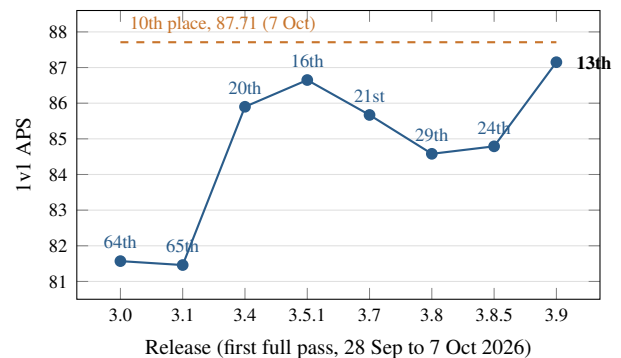


Figure 6: 1v1 RoboRumble APS and rank for each release's first full pass. The dip at 3.8 is the DrussGT route's cost against the weak field; 3.9 recovers it.

6 One turn of the thread: 3.9

The last release shows the loop at full size, and where the human sits in it.

- Read.** 3.8.5's pass left Hadur 1.1 APS below 3.4, mostly in lost rounds against weak opponents. The agent listed the 40 bots that had beaten 3.4 in both passes.
- Bisect.** Whole releases (3.4, 3.5.1, 3.7, 3.8, 3.8.5) were benched in pairs on those bots. The only resolved step was 3.4 to 3.5.1: -6.4 points, with 14 of 39 opponents down after a Holm correction. A 5 Whys record noted that the set was chosen on 3.4's luck, so the size of the loss was uncertain even if its place was not.
- Cause.** 3.5's rammer escape (RAM-2) ran from robots that

Table 2: The plans, in order. Dates are 2026. PR numbers are those of the public repository.

Plan	Stages	Dates	What it delivered	Release
Hadur 2	S0–S7	26–27 Sep	Headless bench, hexagonal core, energy ledger, opponent memory, recognise-and-adapt, press-when-ahead, bullet shadows, tick budget (PRs #18–#32)	2.1, 2.2
Melee extension	M0–M6	27–28 Sep	Pinned duel, posture gate, melee radar and shot reading, minimum-risk movement with virtual bullets, learning field gun, melee memory	3.0
Rumble climb	R0–R9	28 Sep–3 Oct	Paired bench and stratified APS estimate, client reliability, full power vs weak bots, one-JVM session bench, skipped-turn fix, rammer escape (PRs to #82)	3.1–3.5.1
Architecture evolution	A0–A5	5 Oct	Charter, role resolver, World, team ports and link, team baseline (PRs #87–#96)	3.6, 3.7
DrussGT route, Team	D1–D5, T1	5–6 Oct	Power by the lead, light-bullet gun, shadow-weighted aim, shield list; teammates that stop colliding (PRs #98–#105)	3.8
Leak repair	MATCH, RAM-3	6–7 Oct	Route scoped to duels not already won; rammer escape chosen per opponent (PRs #116, #130)	3.8.5, 3.9

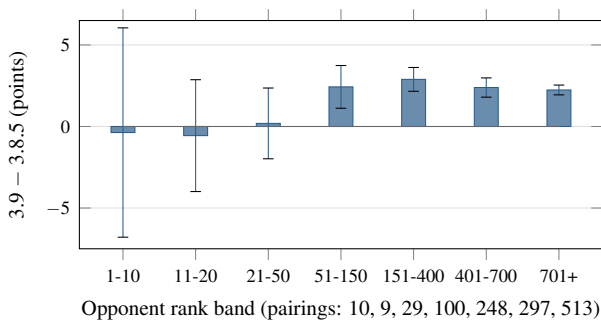


Figure 7: 3.9 minus 3.8.5 per live pairing, by opponent rank band (95% interval). The gain is even across the field below rank 50 and comes from survival. The paired bench gate on 32 of these opponents read level.

confirm as rammers but are really close-range fighters, and running, Hadur was hit two to seven times as often.

- Requirement.** RAM-3, state-driven: *while* the enemy is a confirmed rammer, each round *shall* play one arm from its start, the escape for two rounds, then whichever arm has the higher mean energy margin.
- Build and gate.** One class, `EscapeTrial`, a tagged scenario, a telemetry record per choice. The gate ran on the human’s PC for the CPU: +5.46 on the live losers (ten up after Holm), level on the leak set and the top 20.
- Decide.** The agent offered ship or hold on a decision card, with its estimate: +0.2 to +0.4 APS. The human shipped.
- Live.** 87.15 APS, 13th: +2.32 APS, even across every band below rank 50 (Fig. 7). The estimate was low by a factor of six, and the bench on a wider set had read level. Both facts went into the ledger, with an open question: what in live play makes a rammer confirm on an ordinary bot?

7 Insights

1. The bench is necessary, and it is not the ladder. Three times the bench and the ladder disagreed, and each time the ladder was right: the 3.0 slide, the 3.8 leak, and 3.9’s gain, where the bench gate read -0.02 points on the 32-bot leak set and the live pass read $+2.14 \pm 1.04$ on the same bots. On

Table 3: Three times the bench and the ladder disagreed. Each time the ladder was right, and the fix was to the bench or to what it was asked.

Release	Bench said	Ladder said
3.0, 3.1	35/35 rounds on the worst live rows, at every CPU setting	Slide of 5 to 8 APS within hours; cause: a save scanning a growing data dir
3.8	Top 10 +7.2; weak set level	29th: top 10 +5.2, ranks 21+ -1.55 per pairing
3.9	Leak set level (-0.02)	Same bots $+2.14 \pm 1.04$; field +2.3 APS

the top of the ladder, by contrast, the bench predicts live play well (correlation 0.96 over 18 of the top 19). The lesson is about selection, not tooling. Benches are built from the opponents one is thinking about, and the rating is a flat mean over everyone. A human who reads the whole ladder, band by band, finds what the bench was not built to see.

2. Ground truth turns tuning into engineering. The most productive bench feature was not a score. It was a per-turn log of the engine’s true positions and bullets, against which every inference could be checked. That turned “the surfing feels better” into “13.6% false waves against Crazy, because the engine accelerates before its wall check”, and the shadow geometry from 35% to 100%. Agents are very good at closing a measured gap. They are much less good at noticing that a gap exists when nothing measures it.

3. Review by another agent finds real bugs. An automated reviewer read every pull request. Its findings were often correct and often subtle. On one stage it showed that a new power rule’s margin was set so tight that a real 100-shot window could never satisfy it: the rule passed its own tests but could never fire live. On another it caught a ram response whose power override ignored whether the robot could afford the shot. Separating the author and the reviewer of a change matters as much for agents as for people.

4. Agents report misses honestly when the process asks them to. Not every stage met its own measure. D3's light-bullet gun did not raise the hit rate it was built to raise ($+0.03 \pm 0.21$ points), and the record says so; it merged on the stack's combined gate. 3.9's gate predicted $+0.2$ to $+0.4$ APS, and the ladder gave $+2.3$. Writing both numbers down, with the rule that a refuted learning is marked and not deleted, is what lets the next plan start from the truth.

5. Architecture is a prerequisite for speed. Before S1, a change to the robot could only be judged by watching battles. After S1 it could be replayed bit for bit; after A1 it could be confined to one strand by source hashes. Each fixed point the architecture added let the agents make larger changes with less risk. This is the oldest lesson in the field [3], and it applies to agents with more force, because they make many more changes.

6. The agents' failure modes are specific, and checkable. Across the record, the agents' mistakes fell into a few kinds, each of which a process step caught:

- *A rule that cannot fire.* Thresholds that pass unit tests but can never be met by live data (the power rule above). Caught by review against realistic windows.
- *Confident estimates.* Gate-based APS predictions were off by a factor of six in one direction (3.9) and by a sign in the other (3.8). Caught only by the live pass.
- *Selection on luck.* Opponent sets chosen because a release lost to them overstate the loss. Caught by a 5 Whys record and an unselected rerun.
- *Confounded hosts.* One stage's skipped turns rose twenty-fold on a loaded machine (283 against 15). Caught because the report published per-tick timings, not only totals.
- *Stale artefacts.* A build that fails late can leave the previous jar in place, so a bench measures the wrong robot. Caught by checking the jar for the stage's classes before a run.

None of these is specific to agents. All are what a careful student learns to check, and all become more frequent when changes are cheap.

7. The pace changes the bottleneck. Over 100 pull requests merged in twelve days, with 61 commits on the busiest day. At that rate, the scarce resources are the live ladder (one full pass per afternoon), compute for the bench (gates ran for hours, some on the author's own PC), and the human's attention. The plans adapted: a release bundled several stages, and the live pass became the gate that mattered most.

8 The role of the human

Table 4 lists who did what. We draw it from the project's own record (the plan documents, decision cards and the project memory), not from memory of the work.

Intent. Every plan started from a sentence the human wrote, and the best plans started from the best sentences. The architecture brief above fits in four lines. It contains the whole design test (know the role, adapt instantly, no guessing, no overlap, one identity), and the planning agent's job was to make it concrete. When a goal was vague, such as "climb the rumble", the first plan was too.

Table 4: Who did what, over the twelve days.

Human	Agents
Set every goal (top 30, 15, 10) and brief	Wrote the staged plans and EARS rows
Chose pivots: park melee, keep it, add a team	Wrote the code, tests and fixtures
Picked between options on decision cards	Ran and wrote up every bench gate
Entered each release on the ladders; saved the live pages	Parsed the pages; read the pass by band
Ran long benches on a personal PC	Bisected releases; kept the ledger
Set conventions (what goes where, how to report)	Reviewed every PR; fixed findings
Pushed tags; edited the robot's wiki entries	Drafted release notes and docs

Judgement under uncertainty. The agents produced options with evidence. The human chose. Melee was parked on day one to focus on the duel, then kept when a port of the old melee brain turned out to be cheap. 3.9 shipped on a decision card when its gate gained 5.5 points on the opponents that had beaten 3.4 live but read level on the wider sets. These are bets, and the agents framed them well, but the judgement of which risks were worth taking with a public entry stayed with the person whose name is on it.

Acting in the world. Several steps could only be done by a person: entering a jar on the participants page, saving the ladder's pages when a pass completed, editing the robot's country flag on the community wiki, pushing tags, and running the longest benches on a machine with the right CPU. These steps are mundane, and they are also the loop's connection to reality. Without them, the agents would have optimised the bench indefinitely.

Noticing. The 3.8 regression was raised by the human reading the ladder the afternoon it landed. The agents had a passing release check, with a plus of 7.2 points on the top 10. A habit from college Robocode, watching the whole rating and not only the opponents you care about, was what found it.

What did not change. Table 5 maps the skills a first Robocode project teaches onto the moments in this history where they decided the outcome. Every decisive moment used one of them: stating what the robot must do so that it can be checked, separating the brain from the engine so that it can be tested, logging the truth and comparing against it, distrusting a single battle, and reading the ladder. The agents brought enormous throughput and real expertise in the domain's techniques. The human brought the same craft that college taught [13, 10], applied to specifying, measuring and deciding rather than typing.

9 Implications for teaching

If Robocode is still an entry point, the question for a course is what students should now do with it. Our record suggests an answer: keep the robot, change the deliverable. A student

Table 5: Skills a college Robocode project teaches, and where each decided the outcome here.

Skill	Where it decided the outcome
Model the physics exactly	The ledger’s 99.4% shot recovery; the wall check that caused 13.6% false waves
Separate brain from engine	S1’s core made every later change replayable
Respect the turn budget	The 3.0 slide was skipped turns from a save budget
Log the truth, compare	Shadows from 35% to 100% against engine logs
Distrust one battle	Paired seeds, intervals, the 0.56 APS still to find
Read the whole ladder	The 3.8 leak, found by band, not by the gate
Say what it must do	EARS rows that a test must name

who asks an assistant for a wave-surfing robot will get one, as Hadur 1.x shows. What the assistant does not supply is the bench, the requirement IDs, the replay test, or the habit of reading the whole ladder, and those are where every decisive moment in this history came from. A course can make them the graded artefacts: a baseline measurement before any change, one EARS row and one tagged test per behaviour, a paired gate with an interval, and a written reading of a live pass. The companion course built alongside this work, 17 topics and 15 labs with a small teaching robot, follows that structure; its evaluation is future work.

10 Related work

Robocode has a long record in teaching, as a vehicle for AI [8] and for problem-based learning in introductory programming [15]. Code generation has reopened the question of what novices must learn [2], and controlled studies report large speed-ups for individual tasks [16]. Agent benchmarks such as SWE-bench measure single issue resolution [11, 21], and multi-agent frameworks assign software roles to cooperating models [9, 17]. Our setting differs in three ways: the system is long lived and measured against an external, adversarial ladder; the agents work over many stages against a shared requirements baseline; and the human remains in the loop as the owner of goals and of contact with the world. We are not aware of an end-to-end account of that combination with its data public.

11 Threats to validity

This is a single case with a single developer and one family of models, so the account is illustrative, not general. The rumble is noisy: most pairings in a pass are one or two battles, and other entries change while ours do. We report first full passes and intervals throughout, and the band figures use the complete pass. Our attribution of roles relies on the project’s own records, which the agents wrote; we reduce that bias by citing decisions and data, not characterisations. Finally, “twelve days” measures elapsed time, not effort: the agents ran for many hours in parallel, and the benches for longer.

12 Conclusion and next steps

Twenty years on, Robocode is still a good teacher. Rebuilding Hadur with agents took it from 64th to 13th of about 1,216 in twelve days, onto two more ladders, and through an architecture that lets one identity play all three. The steel thread (goal, plan, requirements, implementation, gate, release, live reading) did not change once in that time, and that stability is what let the pace be safe. The work that mattered most was the oldest work in the craft. The agents made it cheap to act on good decisions; they did not make the decisions less necessary.

The open target is the top 10, 0.56 APS away. The live data puts the remaining points in two groups of weak opponents, close-range rammers that take a fifth of the score and simple guns that win rounds they should not, and not at the top. A full version of this paper will report that climb, an effort accounting, and the companion course built from the same history.

References

- [1] Alan Agresti and Brent A. Coull. Approximate is better than “exact” for interval estimation of binomial proportions. *The American Statistician*, 52(2):119–126, 1998.
- [2] Brett A. Becker, Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, James Prather, and Eddie Antonio Santos. Programming is hard – or at least it used to be: Educational opportunities and challenges of AI code generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE)*, pages 500–506, 2023.
- [3] Frederick P. Brooks, Jr. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987.
- [4] Koen Claessen and John Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000.
- [5] Alistair Cockburn. Hexagonal architecture (ports and adapters). <https://alistair.cockburn.us/hexagonal-architecture/>, 2005.
- [6] Steve Freeman and Nat Pryce. *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley, 2009.
- [7] Leigh Griffin. Hadur_Robot: source, requirements, bench reports and data archive. https://github.com/lgriffin/Hadur_Robot, 2026.
- [8] Ken Hartness. Robocode: using games to teach artificial intelligence. *Journal of Computing Sciences in Colleges*, 19(4):287–291, 2004.
- [9] Sirui Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Andrew Hunt and David Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999.

- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [12] Alistair Mavin, Philip Wilkinson, Adrian Harwood, and Mark Novak. Easy approach to requirements syntax (EARS). In *17th IEEE International Requirements Engineering Conference (RE)*, pages 317–322, 2009.
- [13] Pete McBreen. *Software Craftsmanship: The New Imperative*. Addison-Wesley, 2001.
- [14] Mathew A. Nelson and the Robocode contributors. Robocode: build the best, destroy the rest. <https://robocode.sourceforge.io/>, 2001. First released by IBM alphaWorks in 2001; engine 1.9.5.6 used here.
- [15] Jackie O’Kelly and J. Paul Gibson. RoboCode & problem-based learning: a non-prescriptive approach to teaching programming. In *Proceedings of the 11th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE)*, pages 217–221, 2006.
- [16] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv:2302.06590, 2023.
- [17] Chen Qian et al. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [18] RoboWiki community. RoboRumble, MeleeRumble and TeamRumble rankings. <https://rumble.robowiki.net/>. Accessed 28 September to 7 October 2026.
- [19] RoboWiki community. RoboWiki: wave surfing, guess factor targeting and bullet shielding. <https://robowiki.net/>, 2026.
- [20] Durward K. Sobek, II and Art Smalley. *Understanding A3 Thinking: A Critical Component of Toyota’s PDCA Management System*. Productivity Press, 2008.
- [21] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.